

Analyse critique d'un modèle de détection de fraude

Projet détection de fraude

Rapport technique

11 décembre 2025

Table des matières

1	Introduction et contexte	2
2	Jeu de données et construction des variables	2
2.1	Variables initiales	2
2.2	Feature engineering réalisé	2
3	Approche de modélisation	3
3.1	Logique générale et rôle de « méta-modèle »	3
3.2	Choix des modèles supervisés	3
3.3	Traitement du déséquilibre : <code>class_weights</code> et <code>scale_pos_weight</code>	4
3.4	Choix des hyperparamètres	4
4	Résultats quantitatifs	4
4.1	Performances sur le jeu de test à 16,7 % de fraudes	4
4.2	Impact du seuil de décision	5
5	Analyse interprétative par SHAP	5
5.1	Vue globale (summary plots)	5
5.2	Vue locale (force plots)	5
6	Limites et mise en perspective métier	6
6.1	Impact réel des features disponibles	6
6.2	Perspective métier	6
7	Conclusion	6

1 Introduction et contexte

L'objectif de ce travail est de construire un modèle supervisé capable de détecter des transactions frauduleuses à partir du jeu de données *Cifer Fraud Detection Dataset*. Dans le jeu complet, la fraude est un phénomène extrêmement rare (environ 0,13 % des transactions), ce qui rend l'apprentissage et l'évaluation particulièrement difficiles : un classifieur qui prédit toujours « non fraude » obtient déjà une précision globale très élevée, tout en étant totalement inutile en pratique.

Afin de faciliter l'entraînement et l'analyse, nous avons extrait un sous-échantillon de 60 000 transactions composé de toutes les fraudes disponibles et d'un sous-échantillonnage de non-fraudes. Ce sous-ensemble présente une prévalence de fraude artificiellement augmentée à 16,7 % (environ 10 000 fraudes pour 50 000 non-fraudes). Le but n'est pas de reproduire fidèlement la production, mais de disposer d'un cadre plus lisible pour comparer les modèles et comprendre les facteurs explicatifs de la fraude.

2 Jeu de données et construction des variables

2.1 Variables initiales

Le jeu de données brut contient les variables suivantes :

```
step, type, amount, nameOrig, oldbalanceOrg,  
newbalanceOrg, nameDest, oldbalanceDest, newbalanceDest,  
isFraud, isFlaggedFraud.
```

Les colonnes `nameOrig` et `nameDest` sont des identifiants bruts, peu exploitables telles quelles dans ce contexte. Les variables numériques directement utilisables sont donc principalement le temps (`step`), le type de transaction (`type`), le montant (`amount`) et les soldes avant/après des comptes d'origine et de destination.

2.2 Feature engineering réalisé

Nous avons réalisé un travail de *feature engineering* afin d'enrichir l'information à partir de ces variables de base. Parmi les principales variables dérivées, on peut citer :

- **Encodage temporel** : transformation de `step` en composantes cycliques `step_sin` et `step_cos`, et extraction d'une variable `hour`, afin de capturer la nature périodique et la répartition horaire des transactions.
- **Deltas et incohérences de soldes** : différences entre soldes attendus et observés côté origine et destination, telles que `org_diff`, `dest_diff`, `cashout_orgdiff`, ou des ratios de variation (`delta_ratio`), permettant de mesurer les déséquilibres introduits par la transaction.
- **Ratios de montants** : proportion du solde transférée, par exemple `amount_ratio_orig`, `amount_ratio_dest`, `org_ratio`, `dest_ratio`, `amount_out`, qui caractérisent l'intensité relative de la transaction.
- **Scores d'anomalie non supervisés** : application de modèles d'anomalie (*Local Outlier Factor*, *ECOD*, *Isolation Forest*, *One-Class SVM*, *COPOD*) sur les variables numériques, dont les scores sont ajoutés comme nouvelles features (`anomaly_lof`, `anomaly_ecod`, `anomaly_iso`, `anomaly_ocsvm`, `anomaly_copod`).

Toutes ces nouvelles variables restent des fonctions des informations de base (temps, montants, soldes), mais elles en extraient des structures plus fines (périodes spécifiques, comportements atypiques, incohérences comptables).

3 Approche de modélisation

3.1 Logique générale et rôle de « méta-modèle »

La stratégie globale consiste à utiliser les modèles d'anomalie non supervisés (LOF, ECOD, Isolation Forest, One-Class SVM, COPOD) pour produire des **scores d'anomalie** par transaction, puis à entraîner un modèle supervisé qui agrègera :

- les variables de base (montants, soldes, temps) ;
- les variables dérivées (ratios, deltas) ;
- les scores d'anomalie.

Ce modèle supervisé joue alors le rôle de **méta-modèle** : il apprend comment combiner ces différentes sources d'information pour prédire la probabilité de fraude.

Compte tenu de la nature tabulaire des données, des interactions potentiellement non linéaires (ex. montant $\times$ ratio de solde $\times$ score d'anomalie) et du besoin d'explicabilité (SHAP), nous avons choisi des **modèles d'arbres de décision ensemblistes** : Random Forest, XGBoost et CatBoost.

3.2 Choix des modèles supervisés

Trois familles de modèles ont été expérimentées :

Random Forest (baseline) Un premier modèle *Random Forest* a été utilisé comme baseline. Les forêts aléatoires sont robustes, simples à configurer et adaptées aux données tabulaires. Elles permettent de capturer des interactions non linéaires entre variables sans pré-traitement lourd.

Dans notre cas, les performances obtenues (AUC ROC et AUC PR) sont restées très proches de celles des modèles XGBoost et CatBoost. La Random Forest n'apportant pas d'amélioration significative, nous l'avons conservée comme point de comparaison mais n'en faisons pas le modèle principal du rapport.

XGBoost XGBoost est un algorithme de **gradient boosting** sur arbres de décision, particulièrement performant sur des problèmes tabulaires. Il construit une suite d'arbres faibles, chacun corrigeant les erreurs du précédent, ce qui permet de modéliser des frontières de décision complexes.

Les motivations du choix XGBoost sont :

- capacité à gérer des interactions non linéaires et des variables de nature différente ;
- possibilité d'introduire un paramètre `scale_pos_weight` pour tenir compte du déséquilibre de classes ;
- compatibilité avec SHAP (*TreeExplainer*) pour l'interprétation des résultats.

Dans nos expériences, XGBoost est entraîné sur le train ré-échantillonné (avec une proportion plus forte de fraudes), avec un pré-traitement de type *scaling* sur les variables numériques.

CatBoost CatBoost est un autre algorithme de boosting d'arbres, conçu pour bien gérer les variables catégorielles et réduire le sur-apprentissage grâce à des techniques d'ordonnancement et de régularisation spécifiques. Nous utilisons ici essentiellement des variables numériques, mais CatBoost reste très compétitif et s'intègre bien avec SHAP.

Deux configurations ont été testées :

1. **CatBoost avec `class_weights`** : le modèle est entraîné directement sur l'échantillon à 16,7% de fraudes, en pondérant davantage la classe minoritaire (fraude). Le paramètre `class_weights` permet d'assigner un poids plus élevé aux erreurs sur la classe fraude afin que le modèle ne soit pas tenté de toujours prédire la classe majoritaire.

2. **CatBoost sur un train encore plus équilibré** : toutes les fraudes du train sont conservées, et un sous-échantillon de non-fraudes est tiré de manière à obtenir un ratio d'environ 1 fraude pour 5 non-fraudes. Ce modèle est principalement utilisé comme **modèle d'interprétation** pour l'analyse SHAP : il voit davantage de fraudes et apprend mieux les motifs caractéristiques, ce qui le rend utile pour comprendre « ce qui rend une transaction suspecte », même si ses performances de généralisation restent modestes.

3.3 Traitement du déséquilibre : `class_weights` et `scale_pos_weight`

Le déséquilibre entre la classe non-fraude et la classe fraude est un aspect central de ce problème. Nous avons utilisé deux stratégies complémentaires :

- **Ré-échantillonnage des données** : création d'un train ré-équilibré en sous-échantillonnant les non-fraudes de manière à augmenter la proportion de fraudes (par exemple ratio 1 :5). Ceci permet au modèle de voir davantage d'exemples positifs.
- **Pondération des classes dans la fonction de perte** :
 - dans XGBoost, le paramètre `scale_pos_weight` (non utilisé dans la version finale ré-équilibrée, mais testé) permet d'attribuer un poids plus important aux erreurs sur la classe positive ;
 - dans CatBoost, le paramètre `class_weights` est fixé à une valeur du type (1,5) : une erreur sur une fraude compte cinq fois plus qu'une erreur sur une non-fraude dans la fonction de perte.

Intuitivement, ces paramètres incitent le modèle à « prendre plus de risques » sur la classe minoritaire : il accepte de faire quelques erreurs sur les non-fraudes pour mieux détecter les fraudes.

3.4 Choix des hyperparamètres

Les hyperparamètres des modèles (profondeur des arbres, nombre d'itérations, taux d'apprentissage, sous-échantillonnage) ont été choisies de manière pragmatique, en s'appuyant sur :

- des valeurs usuelles pour les arbres de décision (profondeur modérée, par exemple 6–8) ;
- un nombre d'arbres suffisant (plusieurs centaines) pour permettre au boosting de corriger les erreurs successives ;
- un taux d'apprentissage raisonnablement faible (par exemple 0,05 ou 0,1) pour réduire le risque de sur-apprentissage rapide ;
- des paramètres de sous-échantillonnage (`subsample`, `colsample_bytree`) inférieurs à 1 pour introduire de la diversité dans les arbres et améliorer la généralisation.

L'objectif n'était pas de réaliser un tuning extrêmement poussé (type recherche bayésienne), mais d'atteindre une configuration stable et raisonnable, puis d'évaluer les limites du problème avec ces modèles robustes.

4 Résultats quantitatifs

4.1 Performances sur le jeu de test à 16,7 % de fraudes

Sur le jeu de test (prévalence de fraude $p = 0,167$), les principaux résultats observés sont les suivants (valeurs approximatives) :

- **XGBoost (train ré-échantillonné)** :
AUC ROC $\approx 0,54$, AUC PR $\approx 0,19$.
- **CatBoost avec pondération de classes** :
AUC ROC $\approx 0,52$, AUC PR $\approx 0,17$.
- **CatBoost « équilibré » (modèle d'interprétation)** :
Sur son train ré-équilibré, les performances sont très élevées (AUC ROC $\approx 0,96$, AUC PR

$\approx 0,86$), ce qui reflète un problème plus simple et un certain sur-apprentissage.

Sur le test à 16,7 % de fraudes, ce même modèle obtient AUC ROC $\approx 0,53$, AUC PR $\approx 0,19$.

La prévalence de la fraude p constitue la *baseline* naturelle d'un classifieur aléatoire en AUC PR : ici, un modèle qui prédit des scores de manière aléatoire aurait une AUC PR $\approx 0,167$. Les modèles XGBoost et CatBoost font donc légèrement mieux que le hasard, mais de manière limitée.

4.2 Impact du seuil de décision

L'analyse des courbes « précision / rappel » pour différents seuils montre que :

- avec un seuil de décision autour de 0,5, les modèles préfèrent prédire « non fraude » dans la grande majorité des cas, ce qui conduit à un rappel très faible sur la classe fraude ;
- en abaissant le seuil autour de 0,3, il est possible d'obtenir un rappel plus élevé (jusqu'à 0,85–0,90), mais au prix d'une précision qui reste proche de la prévalence ($\approx 0,17$), soit environ une vraie fraude pour cinq à six alertes.

Autrement dit, le modèle permet de *classer* les transactions par risque et d'augmenter le rappel, mais ne parvient pas à dégager une zone de décision avec une précision nettement supérieure à la baseline.

5 Analyse interprétative par SHAP

5.1 Vue globale (summary plots)

Le modèle CatBoost entraîné sur un train ré-équilibré a servi de base à une analyse SHAP, à la fois globale (summary plot) et locale (force plots). Les principaux enseignements sont :

- Les variables les plus contributives sont :
 - les **scores d'anomalie** issus des modèles non supervisés (`anomaly_ocsvm`, `anomaly_lof`, `anomaly_ecod`, `anomaly_iso`, `anomaly_copod`), dont des valeurs élevées augmentent fortement le score de fraude ;
 - les **variables temporelles** encodées de manière cyclique (`step_sin`, `step_cos`) et `hour`, montrant que certaines fenêtres temporelles sont nettement plus associées à la fraude ;
 - les **ratios et incohérences de soldes** (`org_ratio`, `amount_ratio_orig`, `amount_ratio_dest`, `org_diff`, `dest_diff`, `cashout_orgdiff`), qui capturent les déséquilibres entre soldes avant/après et entre comptes d'origine et de destination.

Cette analyse indique que les fraudes sont :

- globalement plus atypiques dans l'espace des variables numériques (scores d'anomalie élevés) ;
- concentrées sur certaines périodes temporelles ;
- associées à des variations de soldes plus extrêmes et moins cohérentes que les transactions normales.

5.2 Vue locale (force plots)

Les *force plots* locaux illustrent, pour une transaction donnée, comment chaque variable contribue à augmenter ou diminuer le score de fraude. Par exemple, pour une transaction prédite comme fortement frauduleuse, on observe typiquement :

- des montants élevés et des ratios de transferts importants ;
- des scores d'anomalie forts sur plusieurs modèles non supervisés ;
- des incohérences marquées sur les soldes d'origine et de destination.

A l'inverse, une transaction jugée non frauduleuse présente souvent des montants et des ratios plus modestes, des scores d'anomalie faibles, et des variations de soldes cohérentes avec une opération normale, ce qui fait basculer le score global vers la classe « non fraude ».

6 Limites et mise en perspective métier

6.1 Impact réel des features disponibles

Malgré le ré-échantillonnage, l'utilisation de modèles avancés (XGBoost, CatBoost) et un travail important de construction de variables (ratios, deltas, scores d'anomalie, encodage temporel), les performances restent modestes :

- AUC ROC autour de 0,52–0,54 ;
- AUC PR légèrement supérieure à la prévalence ($\approx 0,18$ –0,19 contre 0,167).

Ce résultat suggère que la limite principale ne vient plus du choix du modèle, mais de la **quantité d'information contenue dans les variables disponibles**. Toutes les features construites restent des combinaisons d'un même noyau d'information (temps, type, montants, soldes). Pour franchir un cap en termes de performance, il serait nécessaire d'introduire de nouvelles dimensions métier, par exemple :

- historique détaillé des clients (fréquence, habitudes, ancienneté) ;
- informations sur l'appareil et la localisation (adresse IP, pays, canal) ;
- profil du marchand et type de point de vente ;
- séquences de transactions (approche séquentielle ou temps réel) plutôt que des transactions isolées.

6.2 Perspective métier

Du point de vue opérationnel, les modèles construits peuvent toutefois être utiles pour :

- **prioriser** les transactions, en produisant un score de risque et en alimentant une file d'alertes (par exemple les top- K transactions les plus suspectes) ;
- fournir des **explications** aux analystes via SHAP, en mettant en avant les facteurs qui ont conduit à juger une transaction atypique.

En revanche, les résultats obtenus ne sont pas suffisants pour justifier, à eux seuls, un blocage automatique des transactions. En particulier, si l'on revient au scénario réel où la fraude ne représente que 0,13 % des opérations, la précision brute du modèle s'effondre : le nombre de faux positifs deviendrait ingérable pour une utilisation sans revue humaine.

7 Conclusion

En résumé, le travail mené montre que :

- l'utilisation de modèles à base d'arbres (Random Forest, XGBoost, CatBoost), combinée à un important travail de feature engineering et à des techniques de ré-équilibrage, permet d'extraire un signal réel mais faible de fraude ;
- l'analyse SHAP apporte une compréhension qualitative riche des facteurs associés à la fraude (anomalies globales, fenêtres temporelles, incohérences de soldes) ;
- les performances globales restent toutefois limitées par la pauvreté des informations disponibles dans le jeu de données, ce qui constitue le principal goulot d'étranglement.

Ce constat souligne l'importance, dans un contexte métier réel, de travailler autant sur la **qualité et la richesse des données** que sur le choix des algorithmes, et d'adopter des métriques adaptées (AUC PR, precision@K, analyse du coût des faux positifs / faux négatifs) plutôt que de chercher uniquement à optimiser une précision globale trompeuse.